AI ENGINEERING COHORT · DELIVERABLE 2 · WEEK 4

Design Opportunities

Privacy, Safety & Isolation

Abhijeet Hiwale

July 2026

research/week-4/design_opportunities.pdf

Why This Week Got Added

The original plan stopped at three weeks. This one got added because privacy and safety had only ever shown up as a side effect of scanning other components — a stated constraint in Deliverable 1, a namespace-isolation choice in Week 2 — never from actually researching the attack surface directly. Deliverable 4 requires a real threat model, not an assumption that the earlier constraints already cover it. They don't, and this week is largely about finding out how much they don't.

1. Persistent Memory Turns Prompt Injection Into Something Worse

The finding: a standard prompt injection dies when the session ends. A memory injection persists, can activate weeks later, and — across multiple 2026 sources — is described as nearly undetectable, because there's no reliable way for the runtime to tell attacker-written memory apart from legitimate memory. Both arrive through the same write path. One source calls this "trust laundering": a malicious instruction that enters through something that looks harmless — an email, a code comment, a shared document — reads as trusted context the moment it's sitting in memory, purely because of where it's stored, not what it says.

Why this isn't abstract: this is documented against real, current systems, not just papers. Cisco disclosed a finding called MemoryTrap against Claude Code specifically, patched by Anthropic in April-May 2026. This is an active, ongoing category (OWASP formalized it this year as ASI06: Memory & Context Poisoning), not a settled solved problem.

Proposed change: every memory record gets an explicit provenance tag — user-stated, assistant-generated, tool-derived, or retrieved-document — set at write time. Week 1's admission framework (ADD/UPDATE/DELETE/NOOP) already has a natural place to attach this; it's additive, not a redesign. Unverified or low-provenance memories also get a default expiry rather than being trusted indefinitely, which folds directly into Week 1's four-lever consolidation framework as one more trigger for the decay lever.

Cost / benefit: very low cost, since this rides on infrastructure already adopted in Week 1. The benefit is closing a gap that's currently wide open — right now this system's design has no way to distinguish "the user told me this" from "I read this in a document and decided to remember it," and that distinction is exactly what the current threat literature says matters most.

2. Namespace Isolation Alone Isn't Enough

The finding: vector search is inherently probabilistic. Multi-tenancy needs determinism at the boundary. Those two properties don't mix safely by default — in a shared or even namespace-partitioned index, hybrid search and reranking can still produce cross-tenant collisions if tenant filtering isn't enforced as a hard, deterministic step rather than folded into the similarity ranking.

What this changes: Week 2 adopted namespace-based isolation as sufficient at moderate scale. It's necessary, but this week's research makes clear it's not sufficient on its own — tenant filtering needs to happen as a deterministic pre-filter, applied before or independent of ranking, not as something the similarity score is trusted to respect. I'm treating this as an explicit amendment to the Week 2 decision, not a new one.

Cost / benefit: low cost — this is a query-construction discipline, not new infrastructure — for a control that closes a real, demonstrated failure mode (cross-tenant collisions in hybrid search are documented, not hypothetical).

3. Deletion Doesn't Actually Work If It Only Hits the Source Record

The finding: researchers use the term "backflow" for a specific failure — deleting a source memory doesn't remove the influence of summaries or consolidated records already built from it. One survey's framing is blunt: without propagating deletion to every derived artifact, deletion is "only a retrieval edit," not a real deletion.

Why this is the most consequential finding of the week: Week 1 already adopted a four-lever consolidation framework that explicitly merges and summarizes raw memories. That's exactly the mechanism that creates backflow risk, and it wasn't visible as a risk until this week connected the two. Deliverable 6's acceptance check already requires deletion to work across "source storage and retrieval paths" — this week's research means that requirement has to explicitly extend to every derived artifact, or a system could pass a naive deletion test while still leaking the deleted content through a summary that was never touched.

Proposed change: track lineage from every consolidated/derived artifact back to its source records, and cascade deletion through that lineage graph. This is the one idea this week I'm not marking as a clean adopt — it's a real, non-trivial engineering problem (a dependency graph that has to stay accurate as consolidation keeps running), so it's adopted as a hard requirement, with the actual mechanism prototyped rather than committed to blind.

> Full matrix of all six ideas is in *idea_evaluation_matrix.md*. Challenge notes go in *challenge_notes.md* — self-administered, consistent with weeks 1-3, since there's no live review in this cohort's format.